

야구 데이터 분석 및 실시간 중계 대시보드

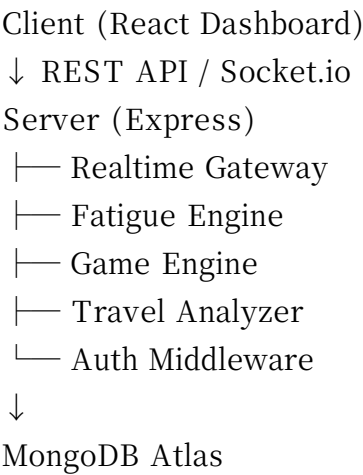
프로젝트 구조 및 초기 설정 제안서

본 문서는 React + Node.js + MongoDB 기반의 야구 데이터 분석 및 실시간 중계 대시보드 프로젝트를 위한 추천 아키텍처와 초기 프로젝트 구조를 정리한 문서입니다.

1. 사용 기술 스택

구분	사용 기술
Frontend	React, Vite
Styling	Tailwind CSS
Backend	Node.js, Express
Database	MongoDB Atlas
Authentication	Firebase Authentication
Realtime	Socket.io
Image Upload	Multer
Deploy	Render

2. 추천 아키텍처



3. 추천 프로젝트 폴더 구조

```
baseball - dashboard/

- └─ apps/
  - | └─ client/
  - | └─ server/

```

```
|— packages/
| |— shared/
| |— fatigue - engine/
| |— realtime - core/
|— docs/
|— .env
|— README.md
```

4. Frontend 구조

```
client/src/
|— api/
|— components/
|— pages/
|— hooks/
|— store/
|— layouts/
|— routes/
|— utils/
```

5. Backend 구조

```
server/src/
|— config/
|— modules/
|— services/
|— engines/
|— sockets/
|— middleware/
|— models/
|— routes/
|— utils/
```

6. 피로도 분석 엔진

입력 데이터 예시:

```
travelDistance: 420km
restHours: 18
consecutiveGames: 5
inningsPlayed: 9
timezoneShift: 1
```

피로도 계산 예시:

fatigue =

(travelDistance * 0.02) +
(consecutiveGames * 8) +
(inningsPlayed * 1.5) -
(restHours * 1.2) +
(timezoneShift * 10)

7. 실시간 중계 구조

경기 데이터 입력 → Express 서버 → Socket.io Broadcast → React Dashboard 실시간 갱신

8. MongoDB 설계 예시

teams

- name
- stadium
- location(lat, lng)

players

- name
- teamId
- fatigueScore

games

- homeTeam
- awayTeam
- realtimeEvents[]

9. 추천 초기 세팅

1) Vite + React 생성

npm create vite@latest client

2) Express 서버 생성

npm init -y

3) 주요 패키지 설치

express, mongoose, socket.io, firebase-admin, multer

10. MVP 개발 순서

1단계

- 로그인
- 실시간 점수판
- 팀/선수 조회

2단계

- 이동 거리 계산
- 휴식 시간 계산
- 피로도 분석

3단계

- 실시간 피로도 변화
- 부상 위험 예측
- 분석 차트 시각화